Implementing a simple Airflow DAG

In this train, we'll learn how to implement a simple Airflow DAG representing a given workflow. We'll become familiar with the practical form and contents of an Airflow DAG script.

Learning objectives

> - Be able to implement a simple Airflow DAG representing a given workflow.
> - Become familiar with the practical form and contents of an Airflow DAG script.
> - Understand atomicity and idempotency in Airflow tasks.

Scenario: Automating Linda's Kitten Image Workflow

Linda loves kittens and uses a script to download daily images from the Unsplash API. However, her workflow has limitations: it's manual, not fault-tolerant, and lacks observability. You've been asked to improve this using Airflow.

Her original steps:

- Query the Unsplash API for "kitten" images
- Save the response as a JSON file
- Parse the JSON to download each image
- Print a notification upon completion

We'll break these into tasks and implement them in Airflow using a DAG.

Defining the Airflow DAG

Here's a simple DAG structure with three tasks:

- `fetch_kitten_urls`: Uses `BashOperator` to call the Unsplash API.
- `get_kitten_images`: Uses `PythonOperator` to download images from JSON.
- `pipeline_notification`: Logs the total number of images downloaded.

1. Module Imports

```
 import airflow
from airflow import DAG
from datetime import timedelta
from airflow.operators.bash_operator import BashOperator
```

```
from airflow.operators.python_operator import PythonOperator
import json, requests
import requests.exceptions as request_exceptions
from pathlib import Path
```

2. DAG Instantiation

```
dag = DAG(
    dag_id='Basic_Airflow_Pipeline',
    start_date=airflow.utils.dates.days_ago(0),
    schedule_interval=timedelta(days=1)
)
```

3. Define Python Task Function

```
def _get_kitten_images(json_file: str, save_path: str):
    with open(json_file, 'r') as jsn_file:
        api_data = json.load(jsn_file)
    img_data_array = []
    for item in api_data:
        try:
            img_data_array.append(requests.get(item["urls"]["small"]).content)
        except request_exceptions.RequestException:
            continue
    save_path = Path(save_path)
    save_path.mkdir(exist_ok=True)
    for i, item in enumerate(api_data):
        img_path = save_path.joinpath(f"{item['id']}.jpg")
        with open(img_path, 'wb') as img_file:
            img_file.write(img_data_array[i])
```

4. Define the Tasks

```
CLIENT_ID = '{your-unsplash-access-key}'

fetch_kitten_urls = BashOperator(
    task_id="fetch_kitten_urls",
    bash_command=f"curl -o /tmp/images_urls.json -L --request GET
'https://api.unsplash.com/photos/random?query=kitten&count=2&client_id={CLIENT_ID}'",
    dag=dag
)

get_kitten_images = PythonOperator(
    task_id="get_kitten_images",
    python_callable=_get_kitten_images,
    op_kwargs={
        "json_file": "/tmp/images_urls.json",
        "save_path": "/tmp/fetched_images/"
    },
```

```
    dag=dag
)

pipeline_notification = BashOperator(
    task_id="pipeline_notification",
    bash_command='echo "There are now $(ls /tmp/fetched_images/ | wc -l) kitten
images."',
    dag=dag
)
```

5. Set Task Dependencies

```
fetch_kitten_urls >> get_kitten_images >> pipeline_notification
```

Conclusion

This train guided you through building a basic DAG in Apache Airflow by automating a simple real-world scenario. The example illustrates how DAGs orchestrate tasks, ensuring reliability and observability through Airflow's architecture.

Key Takeaways ✖

> - **Takeaway 1:** A DAG is a powerful way to structure workflows with task dependencies.
> - **Takeaway 2:** Airflow tasks should follow principles of atomicity and idempotency.
> - **Takeaway 3:** The `BashOperator` and `PythonOperator` are useful for integrating shell commands and Python logic.
> - **Takeaway 4:** The Airflow UI provides strong observability for debugging and monitoring workflows.
> - **Takeaway 5:** Scheduling and retry logic are defined when configuring the DAG and task arguments.